S NO	Title	Date	Sign
1	Performing matrix multiplication and finding eigen vectors and eigen values using TensorFlow	06/01/26	
2	Solving XOR problem using deep feed forward network.	10/01/26	
3	Implementing a deep neural network for performing binary classification tasks.	13/01/26	
4	Using deep feed forward network with two hidden layers for performing multiclass classification and predicting the class.	17/01/26	
5	Using a deep feed forward network with two hidden layers for performing classification and predicting the probability of class.	19/01/26	
6	Using a deep feed forward network with two hidden layers for performing linear regression and predicting values.	27/01/26	
7	Denoising of images using autoencoder.	03/02/26	
8	Implementing regularization to avoid overfitting in binary classification.	10/02/26	
9	Demonstrate recurrent neural network that learns to perform sequence analysis for stock price.	17/02/26	
10	Implementation of convolutional neural network to predict numbers from number images	24/02/26	

Practical No:01

Aim:Performing matrix multiplication and finding eigen vectors and eigen values using TensorFlow

Code:

```
import tensorflow as tf
print("Matrix Multiplication Demo")
x=tf.constant([1,2,3,4,5,6],shape=[2,3])
print(x)
y=tf.constant([7,8,9,10,11,12],shape=[3,2])
print(y)
z=tf.matmul(x,y)
print("Product:",z)
e_matrix_A=tf.random.uniform([2,2],minval=3,maxval=10,dtype=tf.float32,name="matrixA")
print("Matrix A:\n{}\n\n".format(e_matrix_A))
eigen_values_A,eigen_vectors_A=tf.linalg.eigh(e_matrix_A)
print("Eigen Vectors:\n{}\n\nEigen Values:\n{}\n".format(eigen_vectors_A,eigen_values_A))
```

Output:

```
Matrix Multiplication Demo
tf.Tensor(
[[1 2 3]
 [4 5 6]], shape=(2, 3), dtype=int32)
tf.Tensor(
[[ 7  8]
 [ 9 10]
 [11 12]], shape=(3, 2), dtype=int32)
Product: tf.Tensor(
[[ 58  64]
 [139 154]], shape=(2, 2), dtype=int32)
Matrix A:
[[8.552677  4.040143 ]
 [4.240847  4.3108053]]


Eigen Vectors:
[[-0.52569    0.8506762]
 [ 0.8506762  0.52569  ]]

Eigen Values:
[ 1.6901006 11.173381 ]
```

Practical No:02

Aim:Solving XOR problem using deep feed forward network.

Code:

```
import numpy as np
from keras.layers import Dense
from keras.models import Sequential
model=Sequential()
model.add(Dense(units=2,activation='relu',input_dim=2))
model.add(Dense(units=1,activation='sigmoid'))
model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])
print(model.summary())
print(model.get_weights())
X=np.array([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
Y=np.array([0.,1.,1.,0.])
model.fit(X,Y,epochs=100,batch_size=4)
print(model.get_weights())
print(model.predict(X,batch_size=4))
```

Output:

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 2)	6
dense_3 (Dense)	(None, 1)	3

```
 Total params: 9 (36.00 B)
 Trainable params: 9 (36.00 B)
 Non-trainable params: 0 (0.00 B)
None
[array([[-0.01367509,  0.5485312 ],
       [-0.6183617 ,  0.37480056]], dtype=float32), array([0., 0.], dtype=float32), array([[0.7272209],
       [1.1591231]], dtype=float32), array([0.], dtype=float32)]
Epoch 1/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - accuracy: 0.7500 - loss: 0.7456
Epoch 2/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step - accuracy: 0.7500 - loss: 0.7452
Epoch 3/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - accuracy: 0.7500 - loss: 0.7448
Epoch 4/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - accuracy: 0.7500 - loss: 0.7443
Epoch 5/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - accuracy: 0.7500 - loss: 0.7439
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - accuracy: 0.7500 - loss: 0.7162
Epoch 95/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step - accuracy: 0.7500 - loss: 0.7160
Epoch 96/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step - accuracy: 0.7500 - loss: 0.7158
Epoch 97/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - accuracy: 0.7500 - loss: 0.7156
Epoch 98/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - accuracy: 0.7500 - loss: 0.7154
Epoch 99/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - accuracy: 0.7500 - loss: 0.7152
Epoch 100/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step - accuracy: 0.7500 - loss: 0.7150
[array([[-0.01367509,  0.456601  ],
       [-0.6183617 ,  0.28382516]], dtype=float32), array([ 0.        , -0.00015357], dtype=float32), array([[0.7272209],
       [1.069822 ]], dtype=float32), array([-0.09126555], dtype=float32)]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
[[0.47719944]
 [0.5528549 ]
 [0.5979792 ]
 [0.6683417 ]]
```

Practical No:03

Aim: Implementing a deep neural network for performing binary classification tasks.

Code:

```
# ==============================
# Pima Indians Diabetes Prediction
# ==============================
import numpy as np
from numpy import loadtxt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load dataset
dataset = loadtxt('pima-indians-diabetes.csv', delimiter=',')
# Split into input (X) and output (Y)
X = dataset[:, 0:8]
Y = dataset[:, 8]
# Train-test split
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42
)
# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Define the model
model = Sequential()
model.add(Dense(12, input_dim=8, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# Compile the model
model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
# Train the model
model.fit(
    X_train,
    Y_train,
    epochs=150,
    batch_size=10,
```

```
    verbose=1
)
# Evaluate the model
loss, accuracy = model.evaluate(X_test, Y_test, verbose=0)
print(f"Accuracy of model: {accuracy * 100:.2f}%")

# Make predictions (modern replacement for predict_classes)
predictions = (model.predict(X_test) > 0.5).astype(int)

# Show first 5 predictions
print("\nSample Predictions:")
for i in range(5):
    print(
        "Input:", X_test[i],
        "Predicted:", predictions[i][0],
        "Actual:", int(Y_test[i])
    )
```

Output:

```
Epoch 146/150
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8388 - loss: 0.3645
Epoch 147/150
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8355 - loss: 0.3620
Epoch 148/150
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8355 - loss: 0.3621
Epoch 149/150
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8420 - loss: 0.3620
Epoch 150/150
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8355 - loss: 0.3609
Accuracy of model: 74.68%
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step

Sample Predictions:
Input: [ 0.68185612 -0.71402038 -0.61712658  0.81710976  0.93474906  0.26073561
 -0.11637247  0.87809089] Predicted: 1 Actual: 0
Input: [-0.52639686 -0.27664283  0.30191569  0.7522648  -0.70120553  0.48053518
 -0.954231   -1.03594038] Predicted: 0 Actual: 0
Input: [-0.52639686 -0.40160784 -0.29275872 -1.32277365 -0.70120553 -0.15300476
 -0.9245197  -1.03594038] Predicted: 0 Actual: 0
Input: [ 1.28598261 -0.43284909  0.57222224 -1.32277365 -0.70120553 -0.95462672
  1.14932872  0.0950781 ] Predicted: 0 Actual: 0
Input: [ 0.98391937  0.47314726  1.11283533 -1.32277365 -0.70120553 -0.26936924
 -0.77002097  1.48710085] Predicted: 1 Actual: 0
```

Practical No:04

Aim: Using deep feed forward network with two hidden layers for performing multiclass classification and predicting the class.

Code:

```
from keras.models import Sequential
from keras.layers import Dense
from sklearn.datasets import make_blobs
from sklearn.preprocessing import MinMaxScaler
X,Y=make_blobs(n_samples=100,centers=2,n_features=2,random_state=1)
scalar=MinMaxScaler()
scalar.fit(X)
X=scalar.transform(X)
model=Sequential()
model.add(Dense(4,input_dim=2,activation='relu'))
model.add(Dense(4,activation='relu'))
model.add(Dense(1,activation='sigmoid'))
model.compile(loss='binary_crossentropy',optimizer='adam')
model.fit(X,Y,epochs=500)
Xnew,Yreal=make_blobs(n_samples=3,centers=2,n_features=2,random_state=1)
Xnew=scalar.transform(Xnew)
Ynew=model.predict_classes(Xnew)
for i in range(len(Xnew)):
  print("X=%s,Predicted=%s,Desired=%s"%(Xnew[i],Ynew[i],Yreal[i]))
```

Output:

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a laye
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 67ms/step
X=[0.89337759 0.65864154], Predicted=0, Desired=0
X=[0.29097707 0.12978982], Predicted=1, Desired=1
X=[0.78082614 0.75391697], Predicted=0, Desired=0
```

Practical No:05

Aim:Using a deep feed forward network with two hidden layers for performing classification and predicting the probability of class.

Code:

```
from keras.models import Sequential
from keras.layers import Dense
from sklearn.datasets import make_blobs
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# Dataset
X, Y = make_blobs(n_samples=100, centers=2, n_features=2, random_state=1)

# Scaling
scaler = MinMaxScaler()
X = scaler.fit_transform(X)

# Model
model = Sequential()
model.add(Dense(4, input_dim=2, activation='relu'))
model.add(Dense(4, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer='adam')
model.fit(X, Y, epochs=200, verbose=0)

# New data
Xnew, Yreal = make_blobs(n_samples=3, centers=2, n_features=2, random_state=1)
Xnew = scaler.transform(Xnew)

# Predictions
Ynew = model.predict(Xnew)                # probabilities
Yclass = (Ynew > 0.5).astype(int)         # classes

# Output
for i in range(len(Xnew)):
    print(f"X={Xnew[i]}, Predicted_probability={Ynew[i][0]:.4f}, Predicted_class={Yclass[i][0]}")
```

Output:

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 65ms/step
X=[0.89337759 0.65864154], Predicted_probability=0.0417, Predicted_class=0
X=[0.29097707 0.12978982], Predicted_probability=0.9107, Predicted_class=1
X=[0.78082614 0.75391697], Predicted_probability=0.0593, Predicted_class=0
```

Practical No :06

Aim:Aim: Using a deep feed forward network with two hidden layers for performing linear regression and predicting values

Code:

```
from keras.models import Sequential
from keras.layers import Dense
from sklearn.datasets import make_regression
from sklearn.preprocessing import MinMaxScaler
X,Y=make_regression(n_samples=100,n_features=2,noise=0.1,random_state=1)
scalarX,scalarY=MinMaxScaler(),MinMaxScaler()
scalarX.fit(X)
scalarY.fit(Y.reshape(100,1))
X=scalarX.transform(X)
Y=scalarY.transform(Y.reshape(100,1))
model=Sequential()
model.add(Dense(4,input_dim=2,activation='relu'))
model.add(Dense(4,activation='relu'))
model.add(Dense(1,activation='sigmoid'))
model.compile(loss='mse',optimizer='adam')
model.fit(X,Y,epochs=100,verbose=0)
Xnew,a=make_regression(n_samples=3,n_features=2,noise=0.1,random_state=1)
Xnew=scalarX.transform(Xnew)
Ynew=model.predict(Xnew)
for i in range(len(Xnew)):
  print("X=%s,Predicted=%s"%(Xnew[i],Ynew[i]))
```

Output:

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 74ms/step
X=[0.29466096 0.30317302],Predicted=[0.42695668]
X=[0.39445118 0.79390858],Predicted=[0.5843106]
X=[0.02884127 0.6208843 ],Predicted=[0.50276655]
```

Practical No: 07

Aim: Denoising of images using autoencoder.

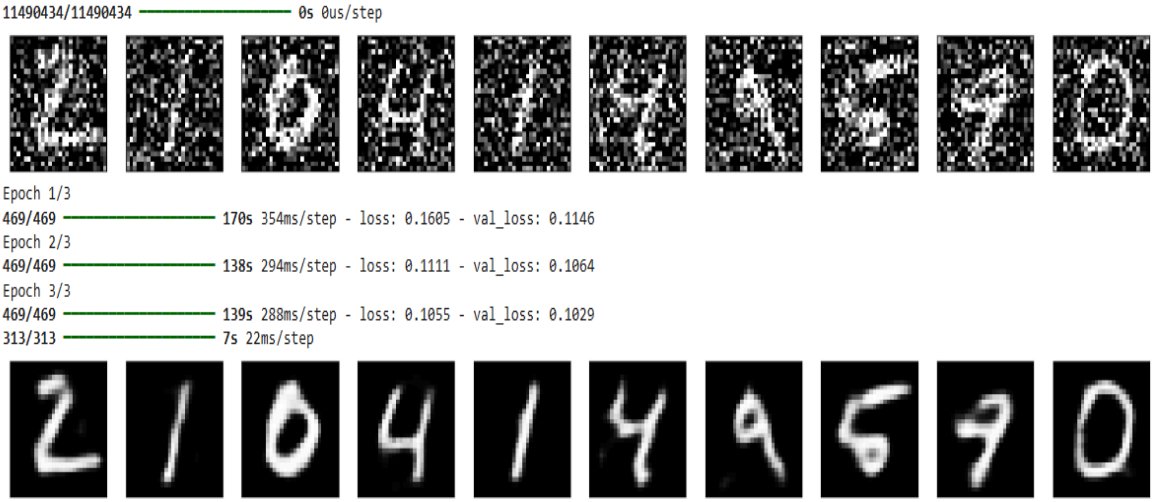
Code:

```
import keras
from keras.datasets import mnist
from keras import layers
import numpy as np
from keras.callbacks import TensorBoard
import matplotlib.pyplot as plt
# Load MNIST dataset
(X_train, _), (X_test, _) = mnist.load_data()
X_train = X_train.astype('float32') / 255.
X_test = X_test.astype('float32') / 255.
# Reshape to (28, 28, 1) for Conv2D input
X_train = np.reshape(X_train, (len(X_train), 28, 28, 1))
X_test = np.reshape(X_test, (len(X_test), 28, 28, 1))
# Add random noise to the images
noise_factor = 0.5
X_train_noisy = X_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=X_train.shape)
X_test_noisy = X_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=X_test.shape)
# Clip values to be between 0 and 1
X_train_noisy = np.clip(X_train_noisy, 0., 1.)
X_test_noisy = np.clip(X_test_noisy, 0., 1.)
n = 10
plt.figure(figsize=(20, 2))
for i in range(1, n + 1):
    ax = plt.subplot(1, n, i)
    plt.imshow(X_test_noisy[i].reshape(28, 28), cmap='gray')
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.show()
input_img = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = layers.MaxPooling2D((2, 2), padding='same')(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
encoded = layers.MaxPooling2D((2, 2), padding='same')(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
x = layers.UpSampling2D((2, 2))(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
```

```
x = layers.UpSampling2D((2, 2))(x)
decoded = layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)
autoencoder = keras.Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
autoencoder.fit(
    X_train_noisy, X_train,
    epochs=3,
    batch_size=128,
    shuffle=True,
    validation_data=(X_test_noisy, X_test),
    callbacks=[TensorBoard(log_dir='/tmo/tb', histogram_freq=0, write_graph=False)]
)
predictions = autoencoder.predict(X_test_noisy)
m = 10
plt.figure(figsize=(20, 2))
for i in range(1, m + 1):
    ax = plt.subplot(1, m, i)
    plt.imshow(predictions[i].reshape(28, 28), cmap='gray')
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.show()
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```



```
Epoch 1/3
469/469 ━━━━━━━━━━━━━━━━━━━━ 170s 354ms/step - loss: 0.1605 - val_loss: 0.1146
Epoch 2/3
469/469 ━━━━━━━━━━━━━━━━━━━━ 138s 294ms/step - loss: 0.1111 - val_loss: 0.1064
Epoch 3/3
469/469 ━━━━━━━━━━━━━━━━━━━━ 139s 288ms/step - loss: 0.1055 - val_loss: 0.1029
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step
```

Practical No: 08

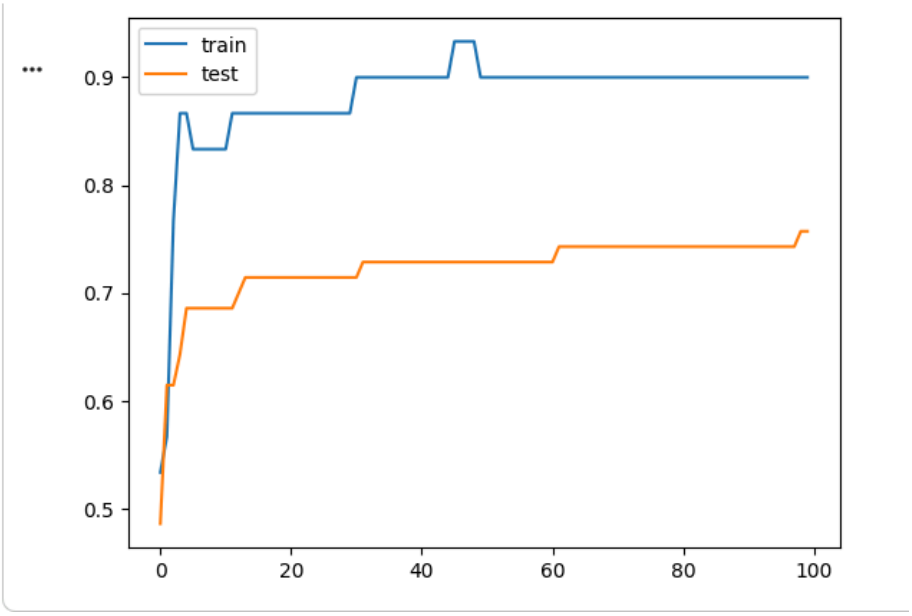
Aim: Implementing regularization to avoid overfitting in binary classification.

Code:

```
from matplotlib import pyplot
from sklearn.datasets import make_moons
from keras.models import Sequential
from keras.layers import Dense
X,Y=make_moons(n_samples=100,noise=0.2,random_state=1)
n_train=30
trainX,testX=X[:n_train,:],X[n_train:]
trainY,testY=Y[:n_train],Y[n_train:]
#print(trainX)
#print(trainY)
#print(testX)
#print(testY)
model=Sequential()
model.add(Dense(500,input_dim=2,activation='relu'))
model.add(Dense(1,activation='sigmoid'))
model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])
history=model.fit(trainX,trainY,validation_data=(testX,testY),epochs=100)
pyplot.plot(history.history['accuracy'],label='train')
pyplot.plot(history.history['val_accuracy'],label='test')
pyplot.legend()
pyplot.show()
```

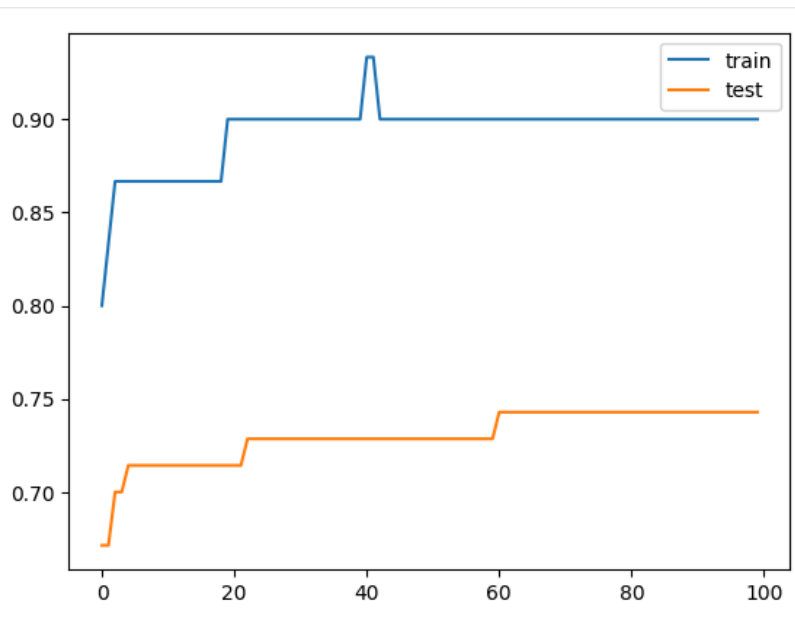
Output:

```
Epoch 1/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.5333 - loss: 0.6807 - val_accuracy: 0.4857 - val_loss: 0.6781
Epoch 2/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 92ms/step - accuracy: 0.5667 - loss: 0.6650 - val_accuracy: 0.6143 - val_loss: 0.6680
Epoch 3/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 85ms/step - accuracy: 0.7667 - loss: 0.6497 - val_accuracy: 0.6143 - val_loss: 0.6583
Epoch 4/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 84ms/step - accuracy: 0.8667 - loss: 0.6349 - val_accuracy: 0.6429 - val_loss: 0.6489
Epoch 5/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 91ms/step - accuracy: 0.8667 - loss: 0.6206 - val_accuracy: 0.6857 - val_loss: 0.6400
Epoch 6/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 89ms/step - accuracy: 0.8333 - loss: 0.6067 - val_accuracy: 0.6857 - val_loss: 0.6314
Epoch 7/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 82ms/step - accuracy: 0.8333 - loss: 0.5934 - val_accuracy: 0.6857 - val_loss: 0.6232
Epoch 8/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 82ms/step - accuracy: 0.8333 - loss: 0.5804 - val_accuracy: 0.6857 - val_loss: 0.6153
Epoch 9/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 81ms/step - accuracy: 0.8333 - loss: 0.5679 - val_accuracy: 0.6857 - val_loss: 0.6078
Epoch 10/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 82ms/step - accuracy: 0.8333 - loss: 0.5557 - val_accuracy: 0.6857 - val_loss: 0.6006
Epoch 11/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 98ms/step - accuracy: 0.8333 - loss: 0.5439 - val_accuracy: 0.6857 - val_loss: 0.5937
```




```
from sklearn.datasets import make_moons
from matplotlib import pyplot
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.regularizers import l2
# Generate dataset
X, Y = make_moons(n_samples=100, noise=0.2, random_state=1)
n_train = 30
trainX, testX = X[:n_train, :], X[n_train:]
trainY, testY = Y[:n_train], Y[n_train:]
# Define model
model = Sequential()
model.add(Dense(
    500,
    input_shape=(2,),
    activation="relu",
    kernel_regularizer=l2(0.001)
))
model.add(Dense(1, activation="sigmoid"))
model.compile(
    loss="binary_crossentropy",
    optimizer="adam",
    metrics=["accuracy"]
)
```

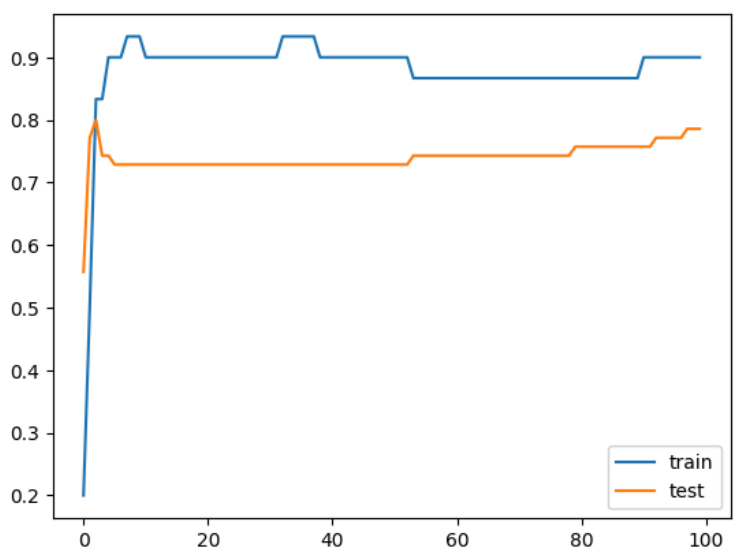
```
# Train model
history = model.fit(
    trainX,
    trainY,
    validation_data=(testX, testY),
    epochs=100,
    verbose=0
)
# Plot accuracy
pyplot.plot(history.history["accuracy"], label="train")
pyplot.plot(history.history["val_accuracy"], label="test")
pyplot.legend()
pyplot.show()
```




```
from matplotlib import pyplot
from sklearn.datasets import make_moons
from keras.models import Sequential
from keras.layers import Dense
from keras.regularizers import l1_l2
X,Y=make_moons(n_samples=100,noise=0.2,random_state=1)
n_train=30
trainX,testX=X[:n_train,:],X[n_train:]
trainY,testY=Y[:n_train],Y[n_train:]
```

```
model=Sequential()
model.add(Dense(500,input_dim=2,activation='relu',kernel_regularizer=l1_l2(l1=0.001,l2=0.001)))
model.add(Dense(1,activation='sigmoid'))
model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])
history=model.fit(trainX,trainY,validation_data=(testX,testY),epochs=100)
pyplot.plot(history.history['accuracy'],label='train')
pyplot.plot(history.history['val_accuracy'],label='test')
pyplot.legend()
pyplot.show()
```

```
Epoch 1/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.2000 - loss: 0.7653 - val_accuracy: 0.5571 - val_loss: 0.7464
Epoch 2/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 93ms/step - accuracy: 0.5000 - loss: 0.7489 - val_accuracy: 0.7714 - val_loss: 0.7358
Epoch 3/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 90ms/step - accuracy: 0.8333 - loss: 0.7329 - val_accuracy: 0.8000 - val_loss: 0.7255
Epoch 4/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 90ms/step - accuracy: 0.8333 - loss: 0.7174 - val_accuracy: 0.7429 - val_loss: 0.7156
Epoch 5/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step - accuracy: 0.9000 - loss: 0.7024 - val_accuracy: 0.7429 - val_loss: 0.7060
Epoch 6/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 88ms/step - accuracy: 0.9000 - loss: 0.6877 - val_accuracy: 0.7286 - val_loss: 0.6967
Epoch 7/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 84ms/step - accuracy: 0.9000 - loss: 0.6734 - val_accuracy: 0.7286 - val_loss: 0.6878
Epoch 8/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 85ms/step - accuracy: 0.9333 - loss: 0.6596 - val_accuracy: 0.7286 - val_loss: 0.6791
```



Practical No: 09

Aim: Demonstrate recurrent neural network that learns to perform sequence analysis for stock price.

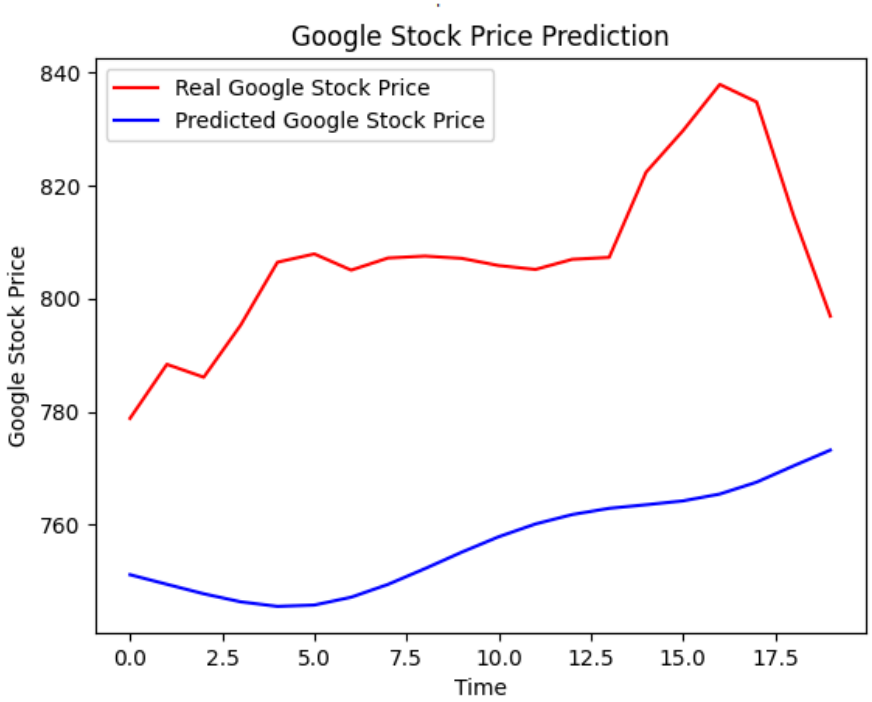
Code:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from keras.models import Sequential
from keras.layers import Dense, LSTM, Dropout
from sklearn.preprocessing import MinMaxScaler
# Load Training Data from GitHub
dataset_train = pd.read_csv(
    'Google_Stock_Price_Train.csv'
)
training_set = dataset_train.iloc[:, 1:2].values  # 'Open' column
# Feature Scaling
sc = MinMaxScaler(feature_range=(0, 1))
training_set_scaled = sc.fit_transform(training_set)
# Create X_train and Y_train dynamically
X_train = []
Y_train = []
for i in range(60, len(training_set_scaled)):
    X_train.append(training_set_scaled[i-60:i, 0])
    Y_train.append(training_set_scaled[i, 0])
X_train, Y_train = np.array(X_train), np.array(Y_train)
# Reshape for LSTM
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
# Build LSTM Model
regressor = Sequential()
regressor.add(LSTM(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
regressor.add(Dropout(0.2))
regressor.add(LSTM(units=50, return_sequences=True))
regressor.add(Dropout(0.2))
regressor.add(LSTM(units=50, return_sequences=True))
regressor.add(Dropout(0.2))
regressor.add(LSTM(units=50))
regressor.add(Dropout(0.2))
```

```
regressor.add(Dense(units=1))
# Compile Model
regressor.compile(optimizer='adam', loss='mean_squared_error')
# Train Model
regressor.fit(X_train, Y_train, epochs=50, batch_size=32)
# Load Test Data from GitHub
dataset_test = pd.read_csv(
    'Google_Stock_Price_Test.csv'
)
real_stock_price = dataset_test.iloc[:, 1:2].values  # 'Open' column
# Prepare Input Data for Prediction
dataset_total = pd.concat((dataset_train['Open'], dataset_test['Open']), axis=0)
inputs = dataset_total[len(dataset_total) - len(dataset_test) - 60:].values
inputs = inputs.reshape(-1, 1)
inputs = sc.transform(inputs)
X_test = []
for i in range(60, len(inputs)):
    X_test.append(inputs[i-60:i, 0])
X_test = np.array(X_test)
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))
# Predict Stock Price
predicted_stock_price = regressor.predict(X_test)
predicted_stock_price = sc.inverse_transform(predicted_stock_price)
# Visualization
plt.plot(real_stock_price, color='red', label='Real Google Stock Price')
plt.plot(predicted_stock_price, color='blue', label='Predicted Google Stock Price')
plt.title('Google Stock Price Prediction')
plt.xlabel('Time')
plt.ylabel('Google Stock Price')
plt.legend()
plt.show()
```

Output:

```
Epoch 1/50
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
  super().__init__(**kwargs)
38/38 ━━━━━━━━━━━━━━━━━━━━ 4s 18ms/step - loss: 0.1054
Epoch 2/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - loss: 0.0058
Epoch 3/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - loss: 0.0053
Epoch 4/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - loss: 0.0050
Epoch 5/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - loss: 0.0055
Epoch 6/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - loss: 0.0057
Epoch 7/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step - loss: 0.0048
Epoch 8/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - loss: 0.0043
Epoch 9/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - loss: 0.0052
Epoch 10/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - loss: 0.0036
Epoch 11/50
38/38 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - loss: 0.0048
Epoch 12/50
```



Practical No: 10

Aim: Implementation of convolutional neural network to predict numbers from number images

Code:

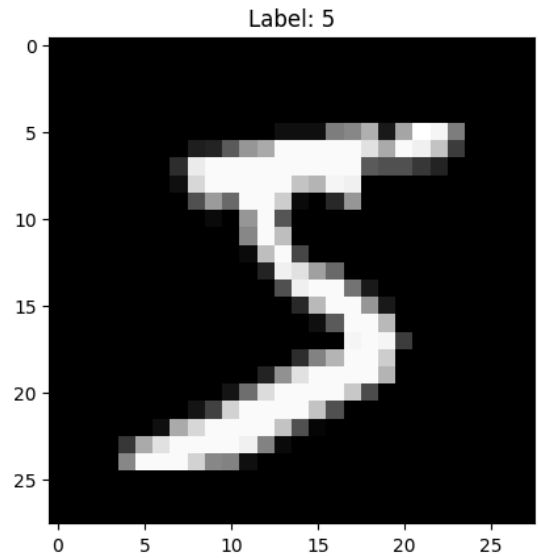
```
!pip install -q tensorflow
# Import required libraries
from keras.datasets import mnist
from keras.utils import to_categorical
from keras.models import Sequential
from keras.layers import Dense, Conv2D, Flatten
import matplotlib.pyplot as plt
# Download MNIST data and split into train and test sets
(X_train, Y_train), (X_test, Y_test) = mnist.load_data()
# Plot the first image in the training dataset
plt.imshow(X_train[0], cmap='gray')
plt.title(f"Label: {Y_train[0]}")
plt.show()
print("Original shape of first image:", X_train[0].shape)
# Reshape the data to include the channel dimension (needed for Conv2D)
X_train = X_train.reshape(60000, 28, 28, 1) / 255.0
X_test = X_test.reshape(10000, 28, 28, 1) / 255.0
# Convert labels to one-hot encoded vectors
Y_train = to_categorical(Y_train)
Y_test = to_categorical(Y_test)
print("One-hot encoded label for first image:", Y_train[0])
# Build the CNN model
model = Sequential([
    Conv2D(64, kernel_size=3, activation='relu', input_shape=(28,28,1)),
    Conv2D(32, kernel_size=3, activation='relu'),
    Flatten(),
    Dense(10, activation='softmax')  # 10 classes for digits 0-9
])
# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
# Train the model
model.fit(X_train, Y_train, validation_data=(X_test, Y_test), epochs=3, batch_size=32)
# Predict the first 4 images in the test set
```

```
predictions = model.predict(X_test[:4])
print("Predictions for first 4 test images:\n", predictions)

# Actual labels for first 4 test images
print("Actual labels (one-hot) for first 4 test images:\n", Y_test[:4])
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```




```
Original shape of first image: (28, 28)
One-hot encoded label for first image: [0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 59s 31ms/step - accuracy: 0.9210 - loss: 0.2611 - val_accuracy: 0.9828 - val_loss: 0.0541
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 58s 31ms/step - accuracy: 0.9856 - loss: 0.0467 - val_accuracy: 0.9849 - val_loss: 0.0486
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 58s 31ms/step - accuracy: 0.9910 - loss: 0.0296 - val_accuracy: 0.9868 - val_loss: 0.0467
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step
Predictions for first 4 test images:
 [[8.7613711e-10 1.6815133e-11 2.8123969e-08 4.4947916e-05 7.7127853e-12
  2.2412643e-11 1.5984653e-16 9.9995494e-01 3.5819252e-08 1.1914111e-07]
 [1.0893862e-09 4.3433616e-08 1.0000000e+00 5.2431423e-12 6.3847778e-14
  1.3408837e-12 6.3063417e-09 2.0287809e-16 1.1543845e-09 6.2143335e-15]
 [1.7848421e-05 9.9795365e-01 5.7894216e-05 1.1153262e-06 4.5456234e-04
  8.1898424e-06 2.3224966e-06 7.4851188e-05 1.4288217e-03 6.7688484e-07]
 [9.9999774e-01 6.6158794e-11 1.1311687e-06 1.4508475e-08 1.6307519e-09
  3.5856399e-08 7.6215736e-07 7.8557392e-08 1.5298218e-08 2.4009992e-07]]
Actual labels (one-hot) for first 4 test images:
 [[0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
```